

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/356980212>

Survey of Software Fuzzing Techniques

Research · December 2021

CITATIONS

0

READS

3,484

3 authors:



[Ashley Barkworth](#)

University of British Columbia

3 PUBLICATIONS 187 CITATIONS

SEE PROFILE



[Lucas McDonald](#)

University of New Brunswick

3 PUBLICATIONS 5 CITATIONS

SEE PROFILE



[Muhammad Ijaz Ul Haq](#)

University of New Brunswick

6 PUBLICATIONS 1 CITATION

SEE PROFILE

Survey of Software Fuzzing Techniques

Lucas McDonald
University of New Brunswick

Muhammad Ijaz Ul Haq
University of New Brunswick

Ashley Barkworth
University of New Brunswick

Abstract—As cybersecurity becomes more than an afterthought and receives the attention it deserves, it is critical to use tools that examine and understand the flaws within programs. One such tool is fuzzing, a testing process which subjects a system or program to a stream of input data, where the goal of the test is to check for exploitable edge cases. This testing is vital, since a hacker may bombard a system with a variety of inputs and scan the system for weaknesses after causing the system to fail. As the security landscape is constantly shifting, there is a need for an up to date review of the state of literature so readers can make informed decisions about the current state of their systems and software. This survey does that by summarizing current state-of-the-art fuzzing approaches, classifying these approaches, and highlighting key insights into the current state of research. The paper also identifies current challenges and suggests future research directions in this area.

I. INTRODUCTION

One stormy night in 1988, Barton Miller, a computer science professor at the University of Wisconsin-Madison, was trying to feed input information into a Unix computer program. These inputs were distorted by electrical noise in thunderstorm, and hence different from what the software required. However, instead of handling the unexpected input gracefully, the program crashed. Miller coined this strategy of evaluating the robustness of programs with unexpected input as “fuzz” testing. Fuzz testing, or fuzzing, is the process of finding exploitable software bugs by randomly feeding invalid, unexpected, or random data as test cases into a target program in order to trigger software errors [1]. Today, fuzzing has become one of the most commonly used methods for discovering software security vulnerabilities. It remains popular due to its conceptual simplicity, low barrier to deployment, and the huge amount of empirical evidence of its effectiveness [2]. For example, in the three years since the ClusterFuzz project launched, it has found about 16,000 bugs in the Chrome browser and about 11,000 bugs in over 160 open source projects [3]. Fuzzing is so effective that it’s now becoming standard in commercial software development processes [4].

A fuzzer takes a software program as input and creates test cases either by mutating a set of seed inputs (which are valid sample test cases) or by generating test cases from scratch using knowledge of the program input’s structure. The fuzzer then executes the program with the input test cases and monitors the program. Any program crashes or errors indicate that the fuzzer has found a bug which could lead to a vulnerability in the program.

Fuzzing has evolved dramatically in the 30 years since it was introduced. Early fuzz testing approaches generated

completely random test inputs, and the only feedback from the program was crashing. Modern fuzzing tools use a variety of approaches to generate input in order to maximize the probability of finding bugs. In this paper, we focus on fuzzing tools that fuzz software programs and applications. We classify fuzzers in three ways. The first way is mutation-based or generation-based, depending on how test cases are created. Second, we classify fuzzers as black box, white box, or grey box, depending on how the program under test is executed. Lastly, while the program under test is executing, a wide array of techniques can be used to monitor and analyze the program execution in order to extract useful information that can improve the fuzzing process. We discuss several of them in our summaries and classify studies by these techniques as well.

The rest of this paper is organized as follows. In Section II, we provide a survey of the current literature on fuzzing and summarize 21 papers that either also surveyed fuzzing or proposed fuzzing techniques. We also present a classification diagram and classify the surveyed techniques in multiple tables. Section III discusses the takeaways of our survey. Section IV highlights six research challenges. Section V highlights six research directions for future researchers. Our conclusion is in Section VI.

II. LITERATURE REVIEW

In this section, we first examine the current state of research, with specific focus on previous surveys of the field. After, we classify fuzzing techniques into three categories: how they generate test cases, how they execute the program, and what techniques they use for monitoring or analysis. See 1 for an illustration of the fuzzing process, which is numbered according to the three ways that we classify fuzzing techniques.

A. Previous Surveys of the State of Research

As we’re producing a product of the state of research as it exists as of publishing, it is pertinent to first examine preexisting research on the state of research and the field itself. This provides an starting point for the analysis of how the field is currently documented. As such we have selected two surveys that provide a survey of the field in depth.

1) State of Issues

Boehme, Cadar, and Roychoudhury [5] produced a survey of the state of research with particular focus on the future

direction of the field and the challenges facing it. Rather than a traditional survey of the field, their work focused on a more top level approach as they surveyed the opinions of both industry experts and researchers on the state of fuzzing. As this was a survey of experts within the field, it produced a general analysis of the field, focusing on guiding future research and analyzing current issues within the field rather than any one subset of fuzzing. The authors provided a brief overview of the basic mechanisms and terminology used within fuzzing before going into detail about the challenges within the field. One notable challenge expressed was the need for better fuzzing methods to find “deep” bugs or bugs that conventional fuzzing methods may not yet find. Another need was for humans to have a greater control over the fuzzing tool, allowing for them to provide greater flexibility and creativity in its usage. Lastly, they found a need for more empirical research. The paper concludes on methods for raising awareness of fuzzing, as the authors believed it was distinctly under-studied. Their ultimate suggestion for greater awareness was to organize capture-the-flag competitions (CTFs) and incorporate fuzzing into the CS education curriculum.

2) State of the Research

In contrast with examining the state of research with a focus on the challenges faced and the opinions of experts within the field, Manes et al. [2] wrote a survey on the research published. The authors focused their survey on attempting to answer some systemic issues the authors believed were becoming prominent in both implementation and research. The authors noticed a disturbing trend in the development of fuzzing techniques - fuzzing software and their design decisions are often poorly documented. The authors pointed out that providing just a simple manual page and the code itself when released open source was leading to a significant loss of information. Issues such as this and the stratification of the language of the field were the main issues the authors sought to address. To correct this, the authors felt it was pertinent to sum up the state of the research as a whole, opting to create a generalized frame work and then place existing research within it. To do this The authors start by proposing some clear definitions for “fuzzing”, “fuzz testing”, “fuzzer”, “fuzz campaign”, “bug oracle” and “fuzz configuration”. The authors proposed a generic algorithm for fuzzing which was generic enough that it can be used in both grey, white or black box implementations. Additionally, they provided a simple framework to understand the processes in which a fuzzer must undergo in order to successfully fuzz a program. The authors, having addressed the generic algorithmic state of research, then compiled a sample of papers from a multitude of research journals with the goal of parsing a representative sample of fuzzing implementations. They then attempted to categorize the papers by sorting and listing the fuzzing implementations by their input type, their box type, and the features they allow, providing a comprehensive guide on which fuzzer of the sampled data does what. These steps were all in the goal

of being able to properly categorize and understand the fuzzing techniques used.

B. Test Case Generation

Generation-based fuzzers use some structural knowledge of the program input, (e.g., input grammar, RFC, file format) to form test cases from scratch. Mutation-based fuzzers take a set of valid initial “seed” inputs and generate test cases by mutating these inputs [6]. See Figure I for the classification of the surveyed fuzzing techniques based on their test generation method.

1) Mutation-based

T-Fuzz (Transformational fuzz) [7] is a mutation-based fuzzer which overcomes the issues faced by mutation-based fuzzers regarding sanity checks. These sanity checks are a big hurdle in attaining code coverage in many proposed fuzzing techniques. If fuzzing programs cannot pass these checks, then an in-depth code analysis cannot be achieved [7]. The general approach for constructing input in smart fuzzing algorithms is to prepare/mutate inputs to penetrate deeper into the software code execution path and whenever this process is obstructed by a hard sanity check, the algorithm searches through complex test cases to pass that check. In T-fuzz, instead of relying solely on the mutation of inputs, the technique mutates the software itself, which means instead of passing complex checks, it simply bypasses these checks by disabling them. Hence, T-Fuzz can penetrate deeper into the software. TFuzz contains four modules. First, it uses an already implemented coverage-based fuzzer to generate inputs and to assess where it is stuck at the check. Second, the program transformer traces out the non-critical checks in the program and based on these checks, makes several copies of the program after removing them. Third, the crash analyzer detects transformed program crash and record inputs and verifies these inputs using symbolic execution analysis. The fourth part is the false-positive filter, which filters out true bugs by using symbolic execution analysis. [7] T-Fuzz was tested on the CGC and LAVA-M datasets and 4 real applications (pngfix, tiffinfo, magick, and pdftohtml).

2) Generation-based

Jitsunari and Arahori [8] developed a generation-based fuzzing approach that leverages machine learning to learn an input grammar for grammar-based fuzzing. This avoids the problem of having to create the grammar manually, which can be costly. The developed method begins by training a neural network using hundreds of sample files (e.g. PDFs). Using the learnt model, their method generates character-sequence page streams that are used to create new files. These new files are fed into the target program, and files that generate a high amount of code coverage are fed back into the model in order to generate more test files that cover a lot of code. The results of their experiments found that their machine learning-based

approach achieved better code coverage than comparable state-of-the-art fuzzers.

Another generation-based fuzzer was presented by Eberlein et al. [9]. They introduced EvoGFuzz, a language-independent evolutionary grammar-based fuzzing approach to fuzz JavaScript, CSS3, and JSON programs. EvoGFuzz combines grammar-based fuzzing and evolutionary computing and aims to generate “interesting” and “complex” test cases which are more likely to reveal defects in a software under test (SUT). EvoGFuzz requires a correctly specified grammar for the target language. It executes the software under test using an individual test input file; the file is deemed “interesting” if it triggers an exception and “complex” based on its structural properties. The “interesting” and “complex” files are selected and used to learn a new grammar that generates more “interesting” and “complex” inputs. When evaluated, EvoGFuzz increased line coverage by 18.43%, 47.93%, and 8.45% for JSON, JS, and CSS3 programs, respectively, and triggered 11 different exception types, including 5 not found by the baseline fuzzer.

JavaScript (JS) engines are an attractive target to attackers since they are a core component of web browsers. Current state-of-the-art fuzzers produce JS test cases that are syntactically correct but may be semantically invalid at runtime. In response, Han, Oh, and Cha [10] developed CodeAlchemist, a generation-based fuzzer using a test case generation technique called semantics-aware assembly which generates JS code snippets that are both syntactically and semantically correct. CodeAlchemist takes a JS engine under test, a set of JS seed files, and a set of user-configurable parameters as input, and outputs a set of bugs found in the engine. First, CodeAlchemist breaks up the JS seeds into combinable building blocks called “code bricks”. Each brick is a valid JS code snippet. The bricks can be annotated with preconditions that define which variables are required before the brick executes. They can also be annotated with post-conditions, which define what variables are defined after executing the brick. Semantics-aware assembly starts with one code brick and repeatedly extends it by merging it with other code bricks. CodeAlchemist uses these assembled code bricks as test cases. Their approach generated 6.8x more semantically valid test cases on average than jsfunfuzz, the current state-of-the-art JS engine fuzzer. It also found 19 bugs in the four major JS engines: ChakraCore, V8, JavaScriptCore, and SpiderMonkey.

C. Program Execution

Black box fuzzers have no knowledge of the internal workings of the program they are testing, and only observe the output of the program execution. Conversely, white box fuzzers can obtain detailed information on the program, including source code, design specifications, and detailed runtime information. This information is then utilized to improve the efficiency of the fuzzing process [24]. Between these two extremes lie grey box fuzzers, which use some information from the program’s execution (e.g., code coverage) but not details. See Figure II for a classification of the surveyed

fuzzing techniques based on how they executed the program under test.

1) Black box

Gopinath et al. [12] proposed a black box fuzzing technique called BFuzzer that quickly generates valid inputs for programs with structured inputs. BFuzzer does not require execution feedback such as code coverage from the target program. Instead, it relies only on failure feedback - in other words, whether the input was valid, and if not, if the input was incorrect or simply “incomplete” (i.e., not long enough). BFuzzer enumerates byte after byte of the program’s input space and tests the program until it finds a valid “prefix”. The tool starts by testing prefixes that are a single byte in length and proceeds to prefixes of two bytes, three bytes, and so on. For each prefix, they iterate over all 255 byte values, append a value, and feed the augmented prefix into the program. If the program returns “complete”, the algorithm has found a valid prefix and exploration ends. If the program returns “incomplete”, the program reinitializes the alphabet and starts testing prefixes that are one byte longer. If the program returns “incorrect”, it discards all possible combinations of the current prefix and the appended byte value. Gopinath et al. evaluated their fuzzer on programs that require complex inputs (e.g., CSV, JSON) and found their approach was more effective at generating inputs than pFuzzer, a comparable white box technique.

Kim et al [25] created a smart seed selection process for black box fuzzing. Black box fuzzing, the authors argued, offers unique and persuasive advantages when compared to both grey box and white box fuzzing. Notably, the authors pointed out that the need for pre-analysis usually cannot be done within the context of security managers, who do not have access to the source code. Thus, white box solutions are not viable. Additionally, the firmware is often heavily versioned and from different sources or manufacturers which the authors argue makes grey box fuzzing not practical. As such, the authors created a black box fuzzer which attempts to make up for the inability of black box fuzzing to monitor code paths, by keeping track of the input, output and time spent. The authors argued that new outputs show a new path was taken and similarly, different completion times show that the path to reach a certain output was different. They use the input as a basis for understanding these measures and for producing similar cases if new outcomes appear to be reached.

2) White box

Ognawala et al. [13] proposed Wildfire, a compositional white box fuzzing technique that uses symbolic execution and fuzzing to find vulnerabilities in C programs and increase coverage in isolated (i.e. parameterized) functions. First, Wildfire creates fuzzing drivers that generate seed inputs given a function’s parameter list and match the inputs generated by the fuzzer to the function arguments. Test cases are then

TABLE I: Overview of Fuzzing Test Case Generation

	Easy to set up?	Prior knowledge	Code coverage	Papers
Generation-based	Hard	Required	High	[8]–[11], [11]
Mutation-based	Easy	None	Depends on initial inputs	[7], [11]–[23]

generated by fuzzing the drivers, and replayed on an instrumented version of the program to generate crash reports. These reports contain details about vulnerabilities exposed by the test cases, such as buffer-overflows, null-pointer dereferences and other index-out-of bounds memory operations. Finally, Wildfire determines the feasibility of the discovered vulnerabilities using stack trace matching and targeted symbolic execution to recursively generate concrete arguments to trigger the vulnerabilities until an entry point is reached.

3) Grey box

Zhang et al. [14] developed BigFuzz, a grey box fuzzer for data-intensive scalable computing (DISC) applications. These applications are difficult to fuzz because they have long latencies, a lot of their source code comes from external frameworks, and random bits or byte mutations cannot generate meaningful data to find real-world bugs. Their approach performs a source-to-source transformation of a DISC application to a semantically equivalent, yet framework-independent program that is more amenable to fuzzing. BigFuzz contains three components that work in concert to make fuzz testing tractable for big data analytics: (1) an abstraction of dataflow implementation using source-to-source transformation with extracted user-defined functions (2) a two level instrumentation and monitoring method for application specific coverage guidance, which monitors coverage for user-defined functions only as opposed to framework code (3) record-level schema-aware input mutations that are geared towards big data analytic errors based on their empirical example. Their experiments found that BigFuzz reduced fuzzing time, improved application code coverage, and achieved a 33-157% improvement in detecting application errors.

Pham et al. [15] developed a smart grey box fuzzer that focused on the structure of file formats - a feature missing from conventional grey box systems - in order to produce more effective fuzzing. With this goal in mind, Pham et al. created a coverage-based grey box fuzzing system that was 'smart', or in other words, is capable of input structure awareness. To achieve this goal, the authors combined several different strategies. The first major change from the conventional grey box implementation is that files are seen as virtual structures rather than as their bit representations. Because of this, the file format is not simply bitwise but rather used as a format to derive information from. This is to address the authors' concerns that the data being produced by conventional grey box implementations fails to take into account the structure of the data rather than just the data as a product of bits. Additionally, the authors sought to maintain performance

levels by weighting seeds based on perceived validity when generating input files. The validity of the file in this case is a measure of whether or not the file is able to pass through the parsing stage of the program. The authors found that their solution was more effective in most situations than the tested grey box measures, with some exceptions.

Zhao et al [16] created a new form of validation for grey box fuzzers, called the format check method (FCM). Rather than allow grey box fuzzers to produce unguided inputs based on their seeds and occasionally overwriting what is aptly described as the "magic bytes" that allow a format to be accepted as that format, Zhao et al. sought to create a means in which these bytes have been maintained so as to avoid creating large amounts of invalid outputs. The authors claim this is different than other approaches as it seeks to ensure good test cases rather than validate existing test cases. As the FCM requires using Naive Bayes classifier to classify the formats, this needs to be trained on data prior to usage. After which, two main areas of a grey box fuzzer are altered: the favourite seed selection and variation. For favourite seed selection, they take formats which are validated against FCM (i.e., they passed). They then take low-cost formats that fail and include them as well to hopefully gain better coverage of edge cases. When generating test cases, they test prior to and after variation, thereby hoping to minimize invalid mutations.

TABLE II: Classification of Fuzzing Techniques Based on Program Execution

	Papers
White box	[8], [13], [17]–[19], [26], [27]
Black box	[12], [25]
Grey box	[16], [19], [20], [28]

D. Monitor/Analysis Techniques

Smart fuzzers collect program control flow and data flow information through program analysis techniques and leverage the collected information to improve themselves. This information is used to generate test cases, explore paths in the program, and guide the fuzzing process. There are several techniques that smart fuzzers use. One is symbolic execution, which treats the program's input as symbols and explores paths in a program through solving path constraints over these symbolic inputs. Similarly, hybrid fuzzing (also called concolic execution) combines concrete execution and symbolic execution, where the execution of the program is initiated with some concrete input and the fuzzer follows the paths that the chosen input lead to. Taint analysis analyzes how a software

application processes an input to determine which parts of the input should be modified [23]. Coverage-feedback fuzzers utilize code coverage information from the program execution in some way. Intelligent fuzzers use machine learning and have the ability to learn, make decisions, and generate inputs intelligently. See the summaries below for fuzzers that use the aforementioned techniques. Figure III provides a classification of our papers based on the techniques used as well as an overview of how each technique can be applied.

1) Symbolic execution

Xu et al. [17] proposed multi-factor potential analysis (MPA), a search strategy for symbolic execution-based fuzzers. The MPA's main goal is to guide fuzzer to choose the path that has more potential of finding a vulnerability. As symbolic execution is a process for resolving constraint issues in a fuzzer, if a constraint is hard to bypass, the fuzzing process will take much more time. Further, if that constraint is at end of the code path, then even solving that constraint will not increase the chance of finding vulnerabilities [17]. MPA solves this issue by assigning weights to unexplored code paths represented in terms of the control flow graph. The weight assigning criteria must be such that it increases the potential for finding vulnerabilities. MPA chooses three key points to call: the high-risk functions, the path covered, and the location of the path. The more the calls to high-risk functions (like malloc, strcpy) in a code path, the greater the chances of a crash. Second, selecting branches that have multiple untraversed branches after it will increase the probability of finding new vulnerabilities. Third, selecting paths that have soft checks that can be easily bypassed will increase code coverage, especially when time is limited [17]. All this results in a better choice of paths covered and increase speed. MPA was tested on the DARPA and CGC benchmark datasets and performed better than tinker and AFL.

2) Hybrid fuzzing

Classical fuzzers are unable to reliably produce outputs to reach specific paths. As such, they have a severe disadvantage when used for deeper testing. Pei-Hong et al. [26] proposed a system that combines the advantages of concolic execution by using symbolic interpretation of logical switches in software to map paths with fuzzing. Though their work is by no means entirely novel, as they point to similar software use a hybrid approach, their work differs in that they were able to produce a more efficient solution [26]. The authors presented their solution of a Priority Based Path Searching Method (PBPS). The importance of efficiency in fuzzing cannot be overstated. As such, their work is extremely relevant. Within a broader context this can be seen as a merger of two different techniques for vulnerability discovery and may represent a broader discussion about each technique, especially as they are combined [26]. The approach put forward by the authors differs from other hybrid approaches in that it takes two key

factors into account: the cost of each missed path and the solving demand of each path [26]. They estimated the cost of missed paths by "calculating the path lengths of the missed paths" [26] while the solving cost is calculated by a "[Simple-hit score] of the missed paths in PBPS" [26]. Because of this approach, the actual implementation is a combination of fuzzing and agnostic concolic technique [26], as it can be applied regardless of the techniques used, making it a very versatile solution [26]. The authors applied PBPS to Driller, a hybrid fuzzer which they then call QuickFuzz. Quickfuzz, compared to standard Driller, had greater code coverage and discovered more vulnerabilities when combined [26]. Though no limitations were stated, the testing was done on a DARPA created resource with hidden vulnerabilities. It's unclear if DARPA created these resources with fuzzing in mind, deliberately creating applications that are designed to fool fuzzing software, in which case the performance benefits may only exist within specific contexts however this is speculation.

Chen et al. proposed SAVIOR [18], another hybrid fuzzing technique testing framework. Hybrid fuzzing leverages fuzz testing to test easy-to-reach code regions and uses concolic execution to explore code blocks guarded by complex branch conditions. Unlike other hybrid testing tools, SAVIOR prioritizes the concolic execution of the seeds that are likely to uncover more vulnerabilities. The system is composed of a fuzzer derived from American Fuzzy Lop (AFL), a concolic executor, and a hybrid coordinator responsible for the orchestration. The fuzzer generates the seed inputs and performs the actual fuzzing. When it barely makes progress, it passes the seeds to the coordinator. The coordinator analyses the program's source code, assessing the reachable code regions and quantifying the amount of vulnerabilities in a chunk of code. Following this analysis, the coordinator computes an "importance" score for each seed, schedules the seeds based on their importance, and feeds them into the concolic executor. Based on coverage feedback from the executor, the coordinator transfers the valuable test cases from the concolic executor back to the fuzzer.

Rustamov et al. [19] noticed that grey box fuzzers are not effective at directed fuzzing. Directed Grey box Fuzzing (DGF) solves this issue but at the expense of stability; additionally, the random nature of their mutations make it difficult to hit complex paths. Similar to the paper by Lin et al. [26], Rustamov et al., rather than creating a directed grey box fuzzer, sought to instead combine a fuzzer with a concolic execution engine. By doing so, they created the target oriented hybrid fuzzer, Bug Miner. Their approach differs from the other hybrid fuzzers in that their implementation uses several key strategies in order to increase performance beyond hybridization. First they implement a machine learning tool to learn from the bug reports created, and create a database which the program then can draw from. Secondly the authors addressed the path explosions problem, by pruning paths unrelated to the specified target. The resulting fuzzer was shown to be more accurate than its competitors as well as much quicker, and seemed to outperform other fuzzers in

every test conducted, of specific note was the time it took to complete a fuzzing. BugMiner took only 9.4 minutes to finish a fuzzing in comparison to 17.2 and 28.6 for ParmeSan and AFLGo using sample data. If this data holds up within a real world context BugMiner could prove to be an invaluable tool in software security,

3) Taint analysis

Chen and Chen [23] developed a mutation-based fuzzer that explores the states of a program using gradient descent and taint analysis. Angora works in a loop. During each iteration of the fuzzing loop, Angora selects an unexplored branch and searches for an input that explores this branch. While exploring the branch, Angora determines which input bytes flow into the corresponding predicate and focuses on mutating these bytes only. After determining which input bytes to mutate, it solves the path constraints on that branch using gradient descent by viewing the path constraint as a function of the program inputs and solving for the inputs that minimizes the function and hence solves the constraint. Once it finds a set of inputs, Angora uses generates test cases from this set by mutating just the relevant input bytes that it found previously. During experimentation, Chen and Chen found that Angora outperformed other state-of-the-art fuzzers by a wide margin.

4) Coverage-feedback

Song et al. [20] found that current fuzzers are not suitable for multi-purpose programs that take both file inputs and command line options (as opposed to just file input). They proposed CrFuzz, a grey box fuzzer which contains a validity checker that uses coverage-feedback and clustering analysis to determine if an input is valid (i.e., will be accepted by a program) or not. CrFuzz mutates file inputs, selects command options, and then checks the validity of this file/command input pair. The pair is saved and mutated again if new code coverage is detected. Invalid inputs are dropped and ignored. Their results found that CrFuzz-augmented fuzzers showed better coverage and found 277 new vulnerabilities in various software programs.

Chen et al. [27] combined multiple fuzzers to develop an integrated, coverage-guided fuzzing technique. They evaluated and defined a variety of different “base” fuzzers on a variety of heuristics, and then implemented a concrete ensemble approach, EnFuzz, that uses seed synchronization to integrate and improve the performance of these base fuzzers. First, the EnFuzz selects a set of base fuzzers. It classifies fuzzers based on their seed mutation/selection strategy, input generation strategy, and coverage information granularity, and then selects a diverse set as possible based on these heuristics. Next, EnFuzz integrates the set of fuzzers in a seed synchronization mechanism. Each base fuzzer maintains a local queue of seeds, while a global pool of seed is maintained by a monitor that identifies “interesting” seeds which can cover new paths or detect unique crashes created from different base fuzzers.

EnFuzz shares those seeds synchronously among all fuzzers. A global map of code coverage is also maintained and updated whenever a seed covers a new code path or branch. They evaluated EnFuzz against six state-of-the-art base fuzzers and found that it consistently outperformed in terms of unique bugs, path and branch coverage with the same resource usage.

Gao et al. [29] presented EM-Fuzz, a fuzzing technique that finds vulnerabilities in embedded system firmware software. It is a binary fuzzer. As there are a variety of architectures in embedded systems, EM-Fuzz is designed to support multi architectural systems, as its implementation is based on previously tested QEUM Emulator. The main focus of EM-Fuzz is continually monitoring memory for memory-related faults like buffer overflow, buffer underflow, buffer un-read, etc. Inputs that trigger these faults are a prime candidate for generating new inputs. This makes EM-Fuzz a coverage and memory-guided technique. EM-fuzz monitors memory and achieves its memory guidance by simply tracking the top memory address of the heap. It designates a pointer “bkr” to the bottom of the heap at the start. As the heap grows, bkr points to the top of the heap, and all the binary instructions that use memory in the range of bottom and bkr are selected for further analysis. Moreover, memory-related functions (like malloc, etc) are also used as branches for coverage guidance. Detecting faults in memory is done by implementing memory detectors that use existing firmware library functions to monitor the memory state of the heap. For every memory-related function, a shadow copy of the heap is made before and after function execution. These copies give a complete picture of all memory addresses and how they are changed. Any corruption is flagged and input to that function is marked as an interesting seed. In this way, EM-fuzz gives a vulnerability testing system with real-time memory checking [29].

InsFuzz [28] is another binary fuzzing technique that targets code coverage by trying to satisfy checks that many other fuzzing algorithms are unable to do. It does this by tracking and evaluating which bytes in the input need to be mutated so it can pass a specific check, for example, if an instruction is comparing a string with the first 8 bytes of an input file, then mutating only the first 8 bytes can increase chances of passing that check and also reduce computing resources and time. Hence, Insfuzz focuses on which part of the input file should be mutated and how to mutate to make fuzzing efficient and effective [28]. Insfuzz’s implementation is based on AFL. To achieve this, Insfuzz incorporates a mechanism for determining which part of the input file is worth mutation. For this it compares the original operand values of a comparison check with each byte mutated value of the input file and provides feedback about how many changed bytes have affected the comparison. This feedback allows the algorithm to find the bytes that are worth mutating and can effectively satisfy specific checks. Once these bytes are specified by the algorithm, it tries to find if the nature of the check, i.e., whether it is a hardcoded or normal check. For hardcoded checks, it uses these hardcoded values to mutate specific bytes in the input file. For normal checks, the algorithm uses bitflip and

arithmetic. The main components of Insfuzz are static analysis, byte inference and fuzzer. The static analysis separates checks that are easy to bypass from difficult ones. Byte inferences finds bytes worth mutating to pass complex checks. Lastly, the fuzzer finds vulnerabilities [28]. InsFuzz was tested on the LAVA-M dataset and programs like size, readelf, nm-new, tiffinfo and jhead.

5) Machine learning

Li et al. [21] proposed V-Fuzz (Vulnerability Prediction-Assisted Evolutionary Fuzzer), a weighted coverage-based binary-oriented fuzzer. Its main objective is to achieve the shortcomings of coverage-based fuzzers. The intuition of coverage-based fuzzers are that more the code is covered, the more vulnerabilities are covered in the program, which is not always right as vulnerable code could be much smaller and still not be detected. Considering this, V-fuzz tries to find vulnerable parts of the program, so the fuzzing process is led in the right direction and time can be saved, as software projects need to be completed in tight schedules. Moreover, it can test programs whose source codes are not available. V-Fuzz at its core is a binary fuzzer that does not require program source code for its operation. It consists of two main components. One is a graph neural network-based vulnerability predictor, which performs the duty of finding vulnerable parts in the supplied software and assign them large weights, but also assigns small weights to the parts which are not deemed vulnerable by the algorithm, in case of some oversight by the vulnerability prediction model. The second component is the fuzzer, which tests the parts of the program according to their weights. These weights depict the importance of that program part in terms of overall program working. These parts are then tested with user-provided inputs and these inputs are mutated intelligently and assigned a score based on the probability of triggering a fault in the tested program. New inputs are generated from previous inputs whose score is higher or have resulted in a program crash. V-Fuzz's core strength is finding vulnerabilities. It does this by identifying binary functions that are vulnerable, using a GNN based algorithm which is trained with vulnerability test data of binary programs. The efficiency of the trained model depends on time and quality (useful parameters are provided). This efficiency results in increased depth and better vulnerability analysis of a program [21]. V-Fuzz was tested on programs Uniq, MP3Gain (Linux), pdftotext, xps document library and libgxps, and found vulnerabilities in ten CVEs.

Many fuzzing techniques try to explore new code paths in the program to increase code coverage. To achieve this, the right input seed is required that can trigger new paths. In this process, a lot of redundant and useless inputs are formed and tested, resulting in a waste of time and resources. So, what if we try to find the correct relation between the code paths and input seeds? This way, we can save a lot of time and computational resources. Finding such a relation is not easy and requires learning. Cheng et al. [22] proposed and implemented a machine learning-based framework to test the

programs that input PDF files. Their idea was to first find the correlation between input PDF files and code execution paths in the program, and then use this to generate proper input seeds that can trigger new execution paths. For this, they used the recurrent neural network-based generator to generate new paths. To train this generator, they recorded the code execution paths. By giving the target program the original input seeds, these paths are stored to form a dataset to train the path generator [22]. Once ready, the path generator will generate new paths given an initial path block. These new paths are used to form a new input seed and transferred to a new PDF input file by the input generator. In this way, comprehensive and effective input seeds that have a high probability of finding vulnerabilities and cover maximum code are formed [22].

Wang et al. [11] presented a deep neural-network-based fuzzer called LAFuzz. This fuzzer can automatically prepare high-quality inputs for programs that take structured or unstructured inputs. LAFuzz incorporated both techniques of generation-based and mutation-based fuzzing to achieve speed and accuracy together. They did this by first collecting data of syntactic inputs from public code repositories and created a training dataset. They used this dataset to train their generative model [11]. This neural network-based generative model, when fully trained, can create new predicted input seeds which are further processed for direct use to fuzz the target program. Finally, LAFuzz starts the mutation-based fuzzing process by using the input seeds from the generative model. In this way, the input seeds can pass lexical and syntactic checks and dig deeper down the code. This model was able to cover 7.55% more code than AFL and was able to detect 30.19% more crashes than AFL [11].

III. DISCUSSION

There are several takeaways from our literature review. First, most researchers preferred mutation-based fuzzers over generation-based fuzzers. Mutation-based fuzzers are fast but have the drawback of generating a large volume of inputs that do not meet syntax requirements and usually do not trigger code execution paths. Generation-based fuzzers can perform better than mutation-based fuzzers but require prior knowledge of the target program's input and incur computational overhead. In addition, the performance of generation-based fuzzers depends on the quality of the input structure specifications. However, the extra knowledge gained by knowing the input structure can result in higher quality test cases and are able to reach deeper parts of the program [6], [30]. Generation-based fuzzing that requires an input grammar is a research challenge in Section IV.

Second, while early fuzzers only used black box program execution, most of the current fuzzing tools used either white or grey box execution. The advantage of grey box and white box fuzzers is that they can improve the fuzzing process by providing feedback after program execution on whether a test case was valid, how deep into the program a test case could reach, and what paths the test case covered. Related to this, the majority of fuzzing techniques collected coverage feedback.

TABLE III: Classification of Papers Based On Techniques Used and Their Applications

Technique	Papers	Test case generation		Program execution	
		Generation-based	Mutation-based	Guiding	Path exploration
Symbolic execution	[7], [13], [17]				✓
Hybrid fuzzing	[18], [19], [26]				✓
Taint analysis	[23]		✓	✓	
Coverage-feedback	[7]–[9], [14], [15], [20], [21], [27], [28], [28], [29]	✓	✓	✓	✓
Machine learning	[8], [11], [16], [21], [22]	✓	✓		

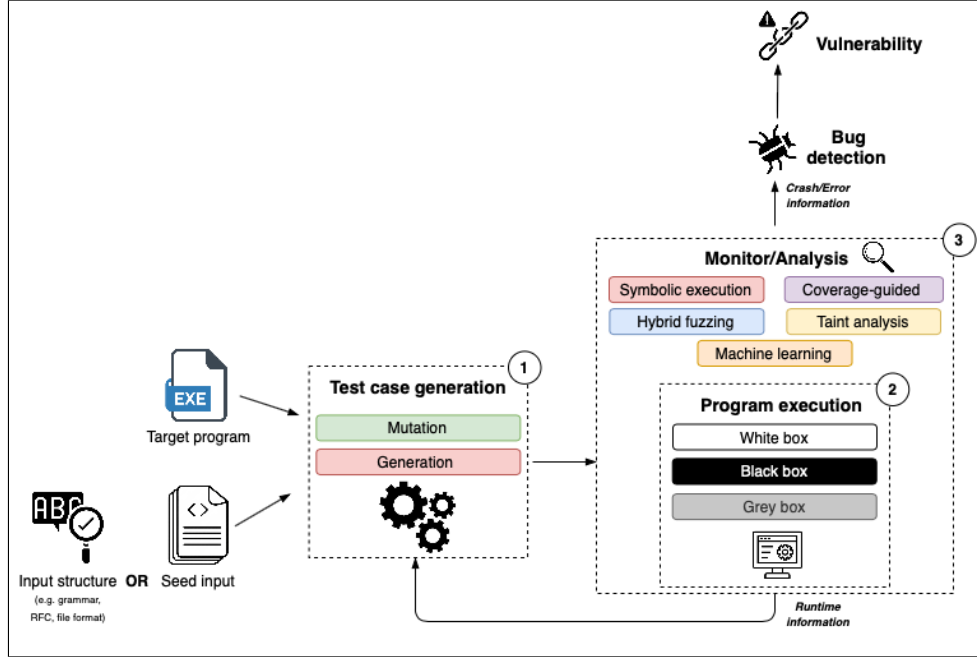


Fig. 1: An overview of the fuzzing process. Fuzzing techniques can be classified by (1) how test cases are generated (2) how the program under test executes (3) what monitoring/analysis techniques are used to get information from the program while it is executing

Third, there are a lot of techniques that can be leveraged to improve fuzzing in all stages. We highlighted the most common techniques, including symbolic execution and taint analysis, but there are many others that researchers have integrated into fuzzers.

Lastly, machine learning has revolutionized the fuzzing process and can be applied to many parts of the fuzzing process, such as test case generation and program path exploration. In Section V, we discuss more about machine learning’s potential in fuzzing as a future research direction.

IV. RESEARCH CHALLENGES

A. Grammar-based Fuzzing

Grammar-based fuzzing uses an input grammar in order to generate syntactically valid input for test cases. For applications that take highly structured inputs, like compilers or

interpreters, a grammar is required in order to generate test cases efficiently. Furthermore, grammar-based fuzzing tools can generate test cases faster while using fewer resources than symbolic execution or other methods [31]. However, there are several limitations with this type of fuzzing. Primarily, grammar-based fuzzing needs an input grammar to generate cases. These grammars are usually created manually and may be unavailable or syntactically incorrect. Grammar-based fuzzing also cannot process unstructured input files like images, and is generally limited to specific file formats. Taking full advantage of grammar-based fuzzing requires overcoming these challenges. For example, to resolve the problem of manually creating a grammar, [8] used machine learning to learn the input grammar automatically.

B. Fuzzing GUI Applications

GUI-based applications are challenging to fuzz for multiple reasons. First, typical fuzzing approaches feed input in order to make an application quit or crash, but for GUI applications, we often don't want to test for crashing. Instead, we want to test for functional correctness, such as displaying the correct information and not losing any user data. Second, many GUI applications are event-driven, where the flow of the program execution depends on a sequence of events, like mouse movement, user input, and button clicks. It is challenging to generate different event sequences and test for concurrency issues in these types of applications, such as atomicity or ordering violations. Given that most users interact with applications through a GUI, it is important to resolve these issues.

C. Absence of Standard Evaluation Criteria

In this survey, we have encountered many fuzzing techniques of different types that focus on different aspects of the target programs to detect vulnerabilities. Some techniques like T-Fuzz and InsFuzz focused on code coverage and considered it an evaluation criterion. Others like EM-Fuzz focused on more vulnerable parts of codes like memory instructions and considered these vital in discovering new bugs and crashes. Some bugs are considered more important than others. Some fuzzers worked better on certain datasets but did not meet the expectations in general software. So, how should we define a fuzzer's success? This is an important question and requires elaborate standard evaluation criteria for fuzzing techniques. We need standardized evaluation criteria for different classes of fuzzers and how they are compared with respective previous work. This evaluation criteria should define what is a reasonable amount of time for fuzzing given a specific program, what is the standard approach for fuzzing the target program, which bugs are more crucial, and so on.

D. Fuzzing Embedded Systems

Embedded systems are rapidly growing, and these systems are now a critical component of the Internet of Things. As these devices are connected to the internet, the vulnerabilities in them are increasing the overall attack surface. Moreover, the application of these embedded systems in high-priority sectors like medical and autonomous vehicles has made the security of these devices of great importance. But these devices do not have fault state detection systems as we have in other computer systems. This results in silent memory corruption which makes it difficult to effectively fuzz these devices [32]. Similarly, many other operational, and resource restrictions limit the fuzzing efficiency to a minimal level. [33] proposed a mechanism to overcome feedback extraction from the fuzzing process on embedded devices which is discussed in the future research section.

E. Language Fragmentation

The state of language fragmentation is a challenge currently within the field. This is a major concern for researchers entering the field as different words often describe the same

concept or phenomenon [2]. Though work has been done to counter this issue, the work presented only functions if used and as such the need to address the issue when writing new research remains present. The authors of this survey suggested using the work of Manes et al. as a starting point for further research, since it provides a working definition of many of the terms used [2]. Additionally, researchers should be considerate to work already published and attempt to use preexisting terms rather than redefine.

F. Lack of Documentation

The state of documentation has posed some concern to researchers, as many implementations provide simply minimal documentation and the source code [2] as a means of reference. This causes a significant loss of information especially in regard to small changes [2] which could lead to novel or best practices being left undocumented. As there is significant demand for fuzzing solutions [5], lack of documentation could be the result of a need to produce working solutions quickly. However, this is unproven. Nevertheless, the authors of this survey stressed the need for documentation beyond simple manual pages, as even the loss of design decisions could lead to repetition or sub optimal solutions in future research and implementation - a fact that should be avoidable in open source implementations.

V. FUTURE RESEARCH DIRECTIONS

A. Machine Learning + Fuzzing

Machine learning can be leveraged at many stages of fuzzing, and many machine learning-based approaches were discussed in our literature review. Input generation is where machine learning is most commonly applied. These approaches use various neural network methods to learn the existing patterns in input files and then identify the likely input forms to trigger new coverage. Similarly, reinforcement learning can learn input grammar for generation-based fuzzers [34]. For the crash analysis stage, machine learning can be utilized to automate the analysis of outputs and crashes generated by fuzzers. It can classify the types of crashes by identifying their root cause or predict whether the reported crashes by a fuzzer are exploitable [35]. Other uses for machine learning include seed generation/selection and mutation operator selection for mutation-based fuzzers. There are several limitations of machine learning. Training requires a large number of samples, supervised learning requires labeled data, and the inputs require to be converted to vectors [24]. Machine learning's potential should inspire researchers to overcome these limitations and incorporate it in even more stages of fuzzing.

B. Fuzzing Cyber Physical Systems

Cyber physical systems (CPSs), such as smart grids and autonomous cars, are systems that tightly integrate their computational and communication components with their physical components. They can operate at a variety of temporal and

spatial scales, may involve human interaction, and often include a variety of sensors to collect information about their surroundings. These systems are frequently used to automate critical public infrastructure, such as water treatment plants or electric power grids. Any disruptions to these systems could cause major damage, and they have become prime targets for attackers, so securing them is vitally important [36]. Fuzzing would be very helpful to identify vulnerabilities in CPSs and is an under-explored area [5]. Blind fuzzing is ineffective due to the enormous search space and resources required. On the other hand, intelligent fuzzing requires impractical amounts of data and network access [37]. All the different interacting processes, temporal and/or spatial scales, and modalities pose a significant challenge to fuzzing CPSs as well. Given the vital role that these systems play in our lives, finding effective ways to fuzz them would be a huge benefit by increasing their security and resiliency.

C. Context-Awareness in Fuzzing

In mutation-based fuzzers, a set of seed inputs are mutated with the aim of discovering more code paths and vulnerabilities. An input that reveals a new code execution path is stored as a seed and used to create new inputs by mutation. As the process goes on, the selected seed list grows quickly, forcing the fuzzing algorithm to choose carefully which seed is used as to increase vulnerability detection and maintain optimal time. For this reason, many researchers have suggested different prioritization proposals like code complexity, as complex code is more vulnerable. So selecting a proper seed is of vital importance in the process of fuzzing. We have also discussed how machine learning is used to create initial seeds that conform to the syntax requirements of the target program but once the syntax is defined the input seeds are mutated and are tested in the target program. The introduction of context-awareness to the fuzzing process can increase the overall productivity of the process. For example, if a fuzzer is aware of the neighboring code around the currently executing code block, and if much of this code is already covered, then mutating such an input seed will not be as beneficial [38]. If the fuzzer is aware of the program's execution path, the type of code block it's currently testing (e.g. memory instructions), or the collections of inputs it needs to execute the target program, it can find more bugs in less time. Different techniques can be wielded to make a fuzzer context-aware. For example, feedback-based algorithms in reinforcement learning can harness context to implement and maximize a reward function for optimal seed selection. Another technique is priority search trees, which can prioritize executing paths based on their length or the number of errors they hit.

D. Concolic Hybridization Fuzzing

One of the main challenges posed by Boehme, Cadar, and Roychoudhury was the need for fuzzing implementations that could reach deeper bugs [5]. One solution to this issue is hybridization fuzzing and concolic testing. The novelty of this trend is clear by the publishing date of each examined

paper, which all were within the last 3 years: 2019 [18], 2020 [19], and 2021 [26]. The main key advantage of this mixed approach is that it provides greater depth traversal while also potentially offering greater efficiency [26]. The hybridization of fuzzers may become the dominant area of research if some of the claims held by its advocates holds true, as the ability to produce more efficient deeper bug analysis of programs and systems which can in turn in some cases be extended into preexisting solutions [19] poses a real threat to shake up the state of research into fuzzing. As such, this is a new and interesting research direction that requires further study.

E. Terminology Consolidation

Though the work of Manes et al. [2] provides a consolidation of techniques and terminology, it already is dated in some ways due to speed of research within the field of fuzzing. An example of which can be seen in all hybrid methods presented. Each were based on grey box fuzzing [19], [26] combined with concolic testing but within the work of Manes et al. it is only addressed within the context of white box testing [39]. As such, it's clear that the state of terminology is still evolving as new methods are introduced and used. Because of this, there is a constant need for surveys of the field.

F. Side-Channel Aware Fuzzing

Side-channel aware fuzzing is a fuzzing technique that uses side-channel analysis to track the success of a fuzzing process [33]. It is important when we need to test embedded or IoT devices, for which source code is not available, and these are many. Internet-connected embedded devices are much more vulnerable to exploits and can be used to trigger faults or introduce malware in connected networks and systems. As these devices don't have adequate resources, a full-scale fuzzing test is not possible. Side-channel aware fuzzing makes white box fuzzing possible for these low resource systems by observing the pattern of power traces a device exhibits on the processing of input. These observations are used to reconstruct code structure with the help of a machine learning algorithm that uses observations to extract features [33]. This technique can be used to test different device software without needing the source code. With the help of reconstructed code, more bugs can be detected in less time. The importance of side-channel aware fuzzing can be realized by just counting the number of embedded devices we encounter and their importance in industry and at home.

VI. CONCLUSION

In conclusion, the state of fuzzing research is a product of the popularity of the technique itself [2]. As such, this survey documents a diverse set of implementations. Because of this, it is necessary to produce works such as this that provide a working overview of current classifications and the techniques used in order to create a starting point for additional research within the field. When examining both the difficulties faced by researchers as well as future challenges, the intent was to provide a guide for future research. Challenges within the

field should be understood as a call to action to remedy an issue, while future research directions should be understood as a suggestion for those seeking to innovate the field, both of which provide substantial benefit to the state of cybersecurity and are in no short supply within the field of fuzzing. Given the popularity of the approach and the many research directions presented, we believe there will be a constant demand for new research within this field within the predictable future.

REFERENCES

- [1] H. Liang, X. Pei, X. Jia, W. Shen, and J. Zhang, "Fuzzing: State of the art," *IEEE Transactions on Reliability*, vol. 67, no. 3, pp. 1199–1218, 2018.
- [2] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, pp. 1–1, 2019.
- [3] M. Böhme, V. Manès, and S. Cha, "Boosting fuzzer efficiency: An information theoretic perspective," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. Association for Computing Machinery, 2020, p. 678–689.
- [4] P. Godefroid, "Fuzzing: Hack, art, and science," *Commun. ACM*, vol. 63, no. 2, p. 70–76, Jan. 2020. [Online]. Available: <https://doi.org/10.1145/3363824>
- [5] M. Boehme, C. Cadar, and A. Roychoudhury, "Fuzzing: Challenges and reflections," *IEEE Software*, vol. PP, 08 2020.
- [6] J. Li, B. Zhao, and C. Zhang, "Fuzzing: a survey," *Cybersecurity*, vol. 1, 12 2018.
- [7] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: Fuzzing by program transformation," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 697–710.
- [8] Y. Jitsunari and Y. Arahori, "Coverage-guided learning-assisted grammar-based fuzzing," in *2019 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2019, pp. 275–280.
- [9] M. Eberlein, Y. Noller, T. Vogel, and L. Grunske, "Evolutionary grammar-based fuzzing," 2020.
- [10] H. Han, D. Oh, and S. Cha, "Codealchemist: Semantics-aware code generation to find vulnerabilities in javascript engines," in *NDSS*, 2019.
- [11] X. Wang, C. Hu, R. Ma, B. Li, and X. Wang, "Lafuzz: Neural network for efficient fuzzing," in *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*, 2020, pp. 603–611.
- [12] R. Gopinath, B. Bendrissou, B. Mathis, and A. Zeller, "Fuzzing with fast failure feedback," 2020.
- [13] S. Ognawala, F. Kilger, and A. Pretschner, "Compositional fuzzing aided by targeted symbolic execution," 2019.
- [14] Q. Zhang, J. Wang, M. A. Gulzar, R. Padhye, and M. Kim, "Bigfuzz: Efficient fuzz testing for data analytics using framework abstraction," in *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 722–733.
- [15] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Caciulescu, and A. Roychoudhury, "Smart greybox fuzzing," *IEEE Transactions on Software Engineering*, 2019.
- [16] S. Zhao, T. Zhou, X. Duan, H. Lin, X. Ying, Q. Shi, and J. Peng, "A format protection method of greybox fuzzing," in *2019 IEEE 19th International Conference on Communication Technology (ICCT)*. IEEE, 2019, pp. 1571–1579.
- [17] L. Xu, W. Dong, L. Yin, and Q. Zhong, "Guide fuzzing with multi-factor potential analysis," in *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2018, pp. 477–482.
- [18] Y. Chen, P. Li, J. Xu, S. Guo, R. Zhou, Y. Zhang, Taowei, and L. Lu, "Savior: Towards bug-driven hybrid testing," 2019.
- [19] F. Rustamov, J. Kim, J. Yu, H. Kim, and J. Yun, "Bugminer: Mining the hard-to-reach software vulnerabilities through the target-oriented hybrid fuzzer," *Electronics*, vol. 10, p. 62, 12 2020.
- [20] S. Song, C. Song, Y. Jang, and B. Lee, "Crfuzz: Fuzzing multi-purpose programs through input validation," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 690–700. [Online]. Available: <https://doi.org/10.1145/3368089.3409769>
- [21] Y. Li, S. Ji, C. Lyu, Y. Chen, J. Chen, Q. Gu, C. Wu, and R. Beyah, "V-fuzz: Vulnerability prediction-assisted evolutionary fuzzing for binary programs," *IEEE Transactions on Cybernetics*, pp. 1–12, 2020.
- [22] L. Cheng, Y. Zhang, Y. Zhang, C. Wu, Z. Li, Y. Fu, and H. Li, "Optimizing seed inputs in fuzzing with machine learning," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2019, pp. 244–245.
- [23] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 711–725.
- [24] Y. Wang, P. Jia, L. Liu, C. Huang, and Z. Liu, "A systematic review of fuzzing based on machine learning techniques," *PLOS ONE*, vol. 15, no. 8, 2020. [Online]. Available: <http://dx.doi.org/10.1371/journal.pone.0237749>
- [25] S. Kim, J. Cho, C. Lee, and T. Shon, "Smart seed selection-based effective black box fuzzing for iiot protocol," *The Journal of Supercomputing*, pp. 1–15, 2020.
- [26] P.-h. Lin, Z. Hong, Y.-h. Li, and L.-f. Wu, "A priority based path searching method for improving hybrid fuzzing," *Computers & Security*, p. 102242, 2021.
- [27] Y. Chen, Y. Jiang, F. Ma, J. Liang, M. Wang, C. Zhou, Z. Su, and X. Jiao, "Enfuzz: Ensemble fuzzing with seed synchronization among diverse fuzzers," 2019.
- [28] H. Zhang, A. Zhou, P. Jia, L. Liu, J. Ma, and L. Liu, "Insfuzz: Fuzzing binaries with location sensitivity," *IEEE Access*, vol. 7, pp. 22 434–22 444, 2019.
- [29] J. Gao, Y. Xu, Y. Jiang, Z. Liu, W. Chang, X. Jiao, and J. Sun, "Em-fuzz: Augmented firmware fuzzing via memory checking," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3420–3432, 2020.
- [30] C. Miller and Z. Peterson, "Analysis of mutation and generation-based fuzzing," 2007.
- [31] H. Salem and J. Song, "A review on grammar-based fuzzing techniques," *International Journal of Computer Science & Security (IJCSS)*, vol. 13, no. 3, p. 114, 2019.
- [32] M. Muench, J. Stijohann, F. Kargl, A. Francillon, and D. Balzarotti, "What you corrupt is not what you crash: Challenges in fuzzing embedded devices," in *NDSS*, 2018.
- [33] P. Sperl and K. Böttinger, "Side-channel aware fuzzing," *CoRR*, vol. abs/1908.05012, 2019. [Online]. Available: <http://arxiv.org/abs/1908.05012>
- [34] Y. Chen, M. Ahmadi, R. M. farkhani, B. Wang, and L. Lu, "Meuzz: Smart seed scheduling for hybrid fuzzing," in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses*, ser. RAID'20, 2020.
- [35] G. Yan, J. Lu, Z. Shu, and Y. Kucuk, "Exploitmeter: Combining fuzzing with machine learning for automated evaluation of software exploitability," in *2017 IEEE Symposium on Privacy-Aware Computing (PAC)*, 2017, pp. 164–175.
- [36] Y. Chen, C. Poskitt, J. Sun, S. Adepu, and F. Zhang, "Learning-guided network fuzzing for testing cyber-physical system defences," *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019. [Online]. Available: <http://dx.doi.org/10.1109/ASE.2019.00093>
- [37] Y. Y. Chen, B. Xuan, C. Poskitt, J. Sun, and F. Zhang, "Active fuzzing for testing and securing cyber-physical systems," *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020. [Online]. Available: <http://dx.doi.org/10.1145/3395363.3397376>
- [38] Y. Li, Y. Xue, H. Chen, X. Wu, C. Zhang, X. Xie, H. Wang, and Y. Liu, "Cerebro: context-aware adaptive fuzzing for effective vulnerability detection," *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019.
- [39] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, 2019.